Atelier Machine Learning

Analyse Comportementale Clientèle Retail

COMPTE RENDU

Atelier Pratique : E-commerce de Cadeaux

Préparé par : MARYEM SIALA

Encadré par : Fadoua Drira

Année Universitaire : 2025-2026

Établissement : École Nationale d'Ingénieurs de Sfax (ENIS)

Table des matières

1 Contexte, Mission et Objectifs **4**
1.1 Contexte & Mission . . . 4
1.2 Objectifs Pédagogiques . . . 4

2 Configuration de l'environnement de travail **5**
2.1 Structure du projet . . . 5
2.2 Environnement virtuel . . . 5
2.3 Gestion des dépendances . . . 6
2.4 Documentation du projet — `README.md` . . . 6
2.5 Versionnement et collaboration — Git / GitHub . . . 6
2.6 Exécution du projet . . . 7

3 Synthèse des features **8**
3.1 Vue d'ensemble . . . 8
3.2 Features numériques transactionnelles (1–17) . . . 8
3.3 Features numériques comportementales (18–34) . . . 9
3.4 Features catégorielles (35–52) . . . 10

4 Exploration des Données (EDA) **12**
4.1 Lecture des données . . . 12
4.2 Structure globale — `df.info()` . . . 12
4.3 Interprétation des types de données . . . 12
4.4 Statistiques descriptives — `df.describe()` . . . 12
4.5 Analyse des valeurs manquantes . . . 13
4.6 Analyse de la variable cible — Churn . . . 13
4.7 Identification des features à traitement spécifique . . . 14
4.8 Analyse de corrélation et Multicolinéarité . . . 14
4.9 Analyse de Corrélation et Multicolinéarité . . . 14

5 Préparation des Données (Data Preprocessing) **15**
5.1 Suppression des features inutiles . . . 15
5.2 Imputation des valeurs manquantes . . . 15
5.3 Traitement des valeurs aberrantes (outliers) . . . 15
5.4 Feature Engineering . . . 16
5.5 Encodage des variables catégorielles . . . 16
5.6 Normalisation — StandardScaler . . . 17
5.7 Analyse de corrélation et multicolinéarité . . . 17
5.8 Réduction dimensionnelle — ACP . . . 18
5.9 Séparation Train / Test stratifiée (80/20) . . . 18
5.10 Rééquilibrage par SMOTE . . . 19
5.11 Récapitulatif du prétraitement . . . 20

6 Modélisation et Entraînement **21**
6.1 Régression Logistique . . . 21
6.2 Random Forest Classifier . . . 21
6.3 Optimisation — GridSearchCV . . . 22

7 Évaluation des Modèles **23**
7.1 Métriques d'évaluation . . . 23
7.2 Comparaison des modèles . . . 23
7.3 Analyse des performances . . . 23
7.4 Matrice de confusion . . . 24

7.5 Importance des variables 24

8 Déploiement de l'Application Flask 25

8.1 Architecture de l'Application 25
8.2 Point d'Entrée et Configuration (run.py) 25
8.3 Patron Application Factory (app/__init__.py) 26
8.4 Module de Prédiction — ChurnPredictor (src/predict.py) 26
8.4.1 Méthode prepare_input() 26
8.5 API REST — Endpoints (app/routes.py) 26
8.6 Sérialisation des Artefacts avec joblib 27
8.7 Lancement de l'Application 27

Résumé

Ce rapport présente la réalisation complète d'un projet de Machine Learning appliqué à l'analyse comportementale des clients d'un site e-commerce de cadeaux. Le travail repose sur un jeu de données contenant 52 variables, caractérisé par la présence de valeurs manquantes, de formats hétérogènes et de valeurs aberrantes.

Une approche structurée a été adoptée pour construire un pipeline de traitement des données de bout en bout. Celui-ci inclut les étapes de nettoyage des données, d'ingénierie des variables (feature engineering), ainsi que la réduction de dimensionnalité à l'aide de l'Analyse en Composantes Principales (ACP).

Deux modèles de classification supervisée ont été implémentés, à savoir la Régression Logistique et la Random Forest. Leurs performances ont été optimisées à l'aide de la méthode GridSearchCV avec validation croisée, puis comparées selon des métriques adaptées, notamment le F1-Score.

L'objectif principal de ce projet est la prédiction du churn (départ des clients), afin d'identifier les profils à risque et d'orienter les stratégies de fidélisation. Enfin, le modèle retenu a été intégré dans une application web développée avec Flask, permettant une utilisation opérationnelle du système de prédiction.

1 Contexte, Mission et Objectifs

1.1 Contexte & Mission

Dans un marché du commerce en ligne de plus en plus concurrentiel, notamment dans le secteur spécialisé de la vente de cadeaux, la compréhension approfondie du comportement client constitue un levier stratégique essentiel pour assurer la croissance et la compétitivité de l'entreprise.

En tant que data scientist au sein de cette entreprise, la mission consiste à exploiter les données issues des historiques transactionnels afin de transformer des données brutes en informations exploitables. Cette démarche vise à soutenir la prise de décision et à améliorer les performances commerciales.

Trois enjeux stratégiques majeurs guident ce projet :

- **Personnalisation de l'offre :** Adapter les stratégies marketing en segmentant la clientèle selon des profils comportementaux précis.
- **Fidélisation des clients (réduction du churn) :** Identifier de manière proactive les clients susceptibles de quitter la plateforme afin de mettre en place des actions de rétention ciblées.
- **Optimisation du chiffre d'affaires :** Concentrer les efforts marketing et commerciaux sur les segments de clients les plus rentables afin de maximiser le retour sur investissement.

Le jeu de données utilisé dans ce projet est volontairement imparfait. Il comporte 52 variables issues de transactions réelles, avec des problèmes de qualité tels que des valeurs manquantes, des formats inconsistants, des valeurs aberrantes (outliers) et des variables inutiles. Cette situation reflète fidèlement les défis rencontrés dans les projets réels de data science.

1.2 Objectifs Pédagogiques

Ce projet vise à développer un ensemble de compétences fondamentales en data science à travers les étapes d'un pipeline de machine learning :

- **Exploration** : analyser la qualité, la structure et la distribution des données afin d'identifier les anomalies, valeurs manquantes et incohérences.
- **Préparation** : nettoyer les données, traiter les valeurs manquantes, encoder les variables catégorielles et normaliser les variables numériques.
- **Transformation** : réduire la dimensionnalité des données via l'Analyse en Composantes Principales (ACP) pour simplifier les modèles et améliorer les performances.
- **Modélisation** : appliquer différents algorithmes de machine learning tels que la régression logistique, les forêts aléatoires, ainsi que des méthodes de clustering et de régression.
- **Évaluation** : interpréter les résultats à l'aide de métriques adaptées (Accuracy, Precision, Recall, F1-score, ROC-AUC) et formuler des recommandations.
- **Déploiement** : développer une interface utilisateur avec Flask permettant l'utilisation opérationnelle du modèle.

2 Configuration de l'environnement de travail

Ce projet a été développé sous **Visual Studio Code (VS Code)** et versionné via **Git / GitHub** afin de garantir la reproductibilité, la traçabilité et la collaboration.

2.1 Structure du projet

L'architecture du projet suit une organisation modulaire adaptée aux projets de machine learning, dans laquelle chaque répertoire remplit un rôle précis dans la chaîne de traitement.

```
E-commerceML/
|-- data/
|   |-- raw/              # Donnees brutes originales (ne pas modifier)
|   |-- processed/        # Donnees nettoyees avant scission
|   \-- train_test/       # Splits 80/20 : X_train, X_test, y_train, y_test
|
|-- notebooks/            # Notebooks Jupyter
|
|-- src/                  # Code Python de production
|   |-- preprocessing.py # Nettoyage, feature engineering, pipeline Scikit-Learn
|   |-- train_model.py   # Entrainement, GridSearchCV, sauvegarde du modele
|   |-- predict.py       # Classe Predictor : chargement et inference
|   \-- utils.py         # Fonctions utilitaires (logging, load/save, plots)
|
|-- models/               # Artefacts serialises (.pkl )
|   |-- preprocessor_pipeline.pkl     # Pipeline de pretraitement
|   |-- best_model.pkl        # Modele final entraine
|   \-- feautures_meta.pkl         # Metadonnees (parametres, metriques)
|
|-- app/                  # Application Flask
|   |-- templates/
|   |   \-- index.html    # Interface HTML/CSS utilisateur
|   |-- __init__.py       # Factory Flask
|   \-- routes.py         # Endpoints REST
|
|-- reports/              # Visualisations et rapports d'evaluation
|-- requirements.txt      # Dependances Python
|-- README.md             # Documentation principale
|-- run.py                # Point d'entree : lancement de l'API Flask
\-- .gitignore
```

Listing 1 – Arborescence du projet `E-commerceML`

2.2 Environnement virtuel

Un environnement virtuel Python est utilisé afin d'isoler les dépendances et de garantir la reproductibilité exacte du projet sur toute machine.

Création de l'environnement

```
python -m venv venv
```

Activation

Windows

```
venv\Scripts\activate

```

Linux / macOS

```
source venv/bin/activate

```

2.3 Gestion des dépendances

Les dépendances du projet sont centralisées dans le fichier `requirements.txt`.

Générer le fichier

```
pip freeze > requirements.txt
```

Installer toutes les dépendances

```
pip install -r requirements.txt
```

Bibliothèques principales

Bibliothèque	Rôle
`pandas`	Manipulation et analyse des données tabulaires
`numpy`	Calcul scientifique et algèbre linéaire
`scikit-learn`	Pipeline ML complet : modèles, métriques, preprocessing
`flask`	Déploiement de l'application web (API REST)
`joblib`	Sérialisation des modèles (`.pkl` / `.joblib`)
`optuna`	Optimisation bayésienne des hyperparamètres
`matplotlib`	Visualisation des données (graphiques, courbes)
`seaborn`	Visualisation statistique avancée (heatmaps, boxplots)

2.4 Documentation du projet — `README.md`

Le fichier `README.md` constitue la documentation principale du projet. Il contient :

- Le titre, la description et l'objectif général (prédiction du churn client)
- La structure complète du projet expliquée répertoire par répertoire
- Les instructions d'installation (environnement virtuel, dépendances)
- Le guide d'utilisation du pipeline de machine learning (3 étapes)
- Un tableau des dépendances avec leurs versions minimales

2.5 Versionnement et collaboration — Git / GitHub

Le projet est géré avec **Git** afin d'assurer un suivi des versions et de permettre la remise du travail via **GitHub**.

Initialisation et premier push

```
git init
git add .
git commit -m "Initial commit"
git branch -M main
git remote add origin <URL_GITHUB>
git push -u origin main
```

2.6 Exécution du projet

toutes les commandes doivent être exécutées depuis la **racine du projet** , avec l'environnement virtuel activé.

3 Synthèse des features

3.1 Vue d'ensemble

Le jeu de données exploité dans ce projet est issu d'une plateforme de vente de cadeaux en ligne. Il regroupe **52 variables** décrivant le comportement transactionnel, démographique et relationnel de chaque client.

Variable cible : `Churn` (feature n° 34) — variable binaire indiquant si le client a quitté la plateforme (`1`) ou est resté actif (`0`). Toutes les autres variables constituent les features explicatives.

La division des 52 features en trois groupes distincts (features numériques transactionnelles, features numériques comportementales avancées, et features catégorielles) reflète la **nature des données**, leur **mode de traitement** , et les **techniques de modélisation** qui leur seront appliquées.

3.2 Features numériques transactionnelles (1–17)

Les 17 premières features décrivent le **comportement d'achat fondamental** de chaque client

N°	Feature	Type	Description	Intervalle	Unité
1	`CustomerID`	Entier	Identifiant unique client	10 000 – 99 999	—
2	`Recency`	Entier	Jours écoulés depuis le dernier achat	0 – 400	jours
3	`Frequency`	Entier	Nombre de commandes distinctes	1 – 50	cmd
4	`MonetaryTotal`	Float	Somme totale dépensée	−5 000 – 15 000	£
5	`MonetaryAvg`	Float	Montant moyen par commande	5 – 500	£
6	`MonetaryStd`	Float	Écart-type des dépenses	0 – 500	£
7	`MonetaryMin`	Float	Dépense minimale enregistrée	−5 000 – 5 000	£
8	`MonetaryMax`	Float	Dépense maximale enregistrée	0 – 10 000	£
9	`TotalQuantity`	Entier	Quantité totale d'articles commandés	−10 000 – 100 000	unités
10	`AvgQtyPerTrans`	Float	Nombre moyen d'articles par commande	1 – 1 000	unités
11	`MinQuantity`	Entier	Quantité minimale commandée	−8 000 – 0	unités
12	`MaxQuantity`	Entier	Quantité maximale commandée	1 – 8 000	unités
13	`CustomerTenure`	Entier	Durée de la relation client	0 – 730	jours
14	`FirstPurchase`	Entier	Ancienneté du premier achat	0 – 730	jours
15	`PreferredDay`	Entier	Jour de semaine favori pour acheter	0 (Lun) – 6 (Dim)	—

(suite du tableau — Features numériques transactionnelles)

N°	Feature	Type	Description	Intervalle	Unité
16	`PreferredHour`	Entier	Heure favorite dans la journée	0 – 23	h
17	`PreferredMonth`	Entier	Mois de l'année favori	1 – 12	—

3.3 Features numériques comportementales (18–34)

Ce second groupe de features numériques capture des dimensions plus **qualitatives du comportement client**. Bien que leur nature reste numérique, elles s'en distinguent par leur **caractère calculé ou dérivé** à partir des transactions brutes.

N°	Feature	Type	Description	Intervalle	Unité
18	`WeekendRatio`	Float	Proportion des achats effectués le weekend	0,0 – 1,0	ratio
19	`AvgDaysBetween`	Float	Délai moyen entre deux commandes	0 – 365	jours
20	`UniqueProducts`	Entier	Nombre de produits différents achetés	1 – 1 000	prod.
21	`UniqueDesc`	Entier	Nombre de descriptions de produits uniques	1 – 1 000	desc.
22	`AvgProdPerTrans`	Float	Produits distincts par commande (moyenne)	1 – 100	prod.
23	`UniqueCountries`	Entier	Nombre de pays associés au compte	1 – 5	pays
24	`NegQtyCount`	Entier	Nombre de lignes à quantité négative	0 – 100	occ.
25	`ZeroPriceCount`	Entier	Nombre de lignes à prix nul	0 – 50	occ.
26	`CancelledTrans`	Entier	Nombre de transactions annulées	0 – 50	trans.
27	`ReturnRatio`	Float	Taux de retour (quantités négatives / total)	0,0 – 1,0	ratio
28	`TotalTrans`	Entier	Total de lignes transactionnelles	1 – 10 000	lignes
29	`UniqueInvoices`	Entier	Nombre de factures distinctes	1 – 500	fact.
30	`AvgLinesPerInv`	Float	Nombre moyen de lignes par facture	1 – 100	lignes
31	`Age`	Float	Âge estimé du client	18 – 81 ou `NaN`	ans
32	`SupportTickets`	Float	Nombre de tickets support ouverts	`-1`, `0-15`, `999`	tickets

(suite — Features numériques avancées)

N°	Feature	Type	Description	Intervalle	Unité
33	`Satisfaction`	Float	Note de satisfaction client	`-1, 0, 1-5, 99`	/5
Variable cible					
34	`Churn`	Binaire	Indicateur de départ client	`0` (fidèle), `1` (parti)	—

3.4 Features catégorielles (35–52)

Les 18 dernières features sont de nature **qualitative** : elles décrivent des profils, des segments, des statuts et des préférences qui ne peuvent pas être représentés par de simples valeurs numériques continues. Leur séparation des features numériques est basée sur le fait que les algorithmes de machine learning ne peuvent pas ingérer des chaînes de caractères directement. Ces variables doivent être transformées en représentations numériques via des techniques d'encodage spécifiques.

N°	Feature	Card.	Description	Modalités	Encodage
35	`RFMSegment`	4	Segment de valeur client	Champions, Fidèles, Potentiels, Dormants	`Ordinal / OHE`
36	`AgeCategory`	7	Tranche d'âge du client	18-24, 25-34, ..., 65+, Inconnu	`Ordinal`
37	`SpendingCat`	4	Niveau de dépense agrégé	Low, Medium, High, VIP	`Ordinal`
38	`CustomerType`	5	Profil comportemental	Hyperactif, Régulier, Occasionnel, Nouveau, Perdu	`One-Hot`
39	`FavoriteSeason`	4	Saison d'achat préférée	Hiver, Printemps, Été, Automne	`OHE / Cyclique`
40	`PreferredTime`	5	Moment de la journée favori	Matin, Midi, Après-midi, Soir, Nuit	`Ordinal / OHE`
41	`Region`	8	Région géographique	UK, Europe N/S/E/C, Asie, Autre	`One-Hot`
42	`LoyaltyLevel`	5	Niveau d'ancienneté	Nouveau, Jeune, Établi, Ancien, Inconnu	`Ordinal`
43	`ChurnRisk`	4	Niveau de risque de départ estimé	Faible, Moyen, Élevé, Critique	`Ordinal`
44	`WeekendPref`	3	Préférence temporelle d'achat	Weekend, Semaine, Inconnu	`One-Hot`

(suite — Features catégorielles)

N°	Feature	Card.	Description	Modalités	Encodage
45	`BasketSize`	4	Taille habituelle du panier	Petit, Moyen, Grand, Inconnu	`Ordinal`
46	`ProdDiversity`	3	Diversité des achats	Spécialisé, Modéré, Explorateur	`One-Hot`
47	`Gender`	3	Genre du client	M, F, Unknown	`One-Hot`
48	`AccountStatus`	4	État du compte client	Active, Suspended, Pending, Closed	`One-Hot`
49	`Country`	37+	Pays de résidence	UK, France, Germany, etc.	`Target Enc.`
50	`Newsletter`	1	Abonnement newsletter (constante)	Yes (unique)	`Suppression`
51	`RegistDate`	Var.	Date d'inscription (formats mixtes)	`"12/03/10"`, `"2010-03-12"`, ...	`Parsing + FE`
52	`LastLoginIP`	Uniq.	Adresse IP dernière connexion	`"192.168.1.45"`, ...	`Feature Eng.`

4 Exploration des Données (EDA)

4.1 Lecture des données

La première opération consiste à charger le dataset et à identifier automatiquement les types de variables. Scikit-Learn distingue les features numériques (entiers et flottants) des features catégorielles (chaînes de caractères), car elles nécessitent des pipelines de prétraitement différents.

4.2 Structure globale — `df.info()`

La méthode `df.info()` est l'outil de diagnostic le plus complet pour une première inspection. Elle permet d'observer les types de données, le nombre de valeurs non nulles ainsi que la présence éventuelle de valeurs manquantes. Cette étape met également en évidence certaines incohérences de typage.

On observe notamment que certaines variables supposées numériques sont stockées sous forme de texte, ce qui indique la présence de valeurs parasites ou de formats mixtes. De plus, certaines colonnes présentent des valeurs manquantes importantes, notamment une variable représentant l'âge des clients.

4.3 Interprétation des types de données

Type pandas	Signification	Exemples dans ce dataset
`int64`	Nombre entier 64 bits	`Recency`, `Frequency`, `TotalQuantity`
`float64`	Nombre décimal	`Age`, `ReturnRatio`, `SatisfactionScore`
`object`	Texte ou type mixte	`MonetaryTotal` (devrait être float — problème de format)
`str`	Chaîne de caractères	`RFMSegment`, `Gender`, `Country`

4.4 Statistiques descriptives — `df.describe()`

Les statistiques descriptives permettent d'obtenir une première compréhension de la distribution des variables numériques. Elles fournissent des informations essentielles telles que la moyenne, l'écart-type, les valeurs minimales et maximales, ainsi que les quartiles.

L'analyse de ces indicateurs met en évidence plusieurs anomalies importantes, notamment :

Feature	count	mean	std	min	max	Anomalie ?
`Recency`	4372	92,0	100,8	1	374	—
`Frequency`	4372	5,1	9,3	1	248	Outlier max
`TotalQuantity`	4372	1122	4673	−303	196 719	Valeurs neg.
`MinQuantity`	4372	−38	1682	−80 995	12 540	Valeurs neg.
`MaxQuantity`	4372	120	1685	−144	80 995	Outlier max
`WeekendRatio`	4372	0,13	0,28	0,0	1,0	—
`ReturnRatio`	4372	0,03	0,11	0,0	1,0	—
`Age`	**3061**	49,1	18,3	18	80	**1 311 NaN**
`SupportTicketsCo`	4372	21,8	139,3	−1		
`SatisfactionScor`	4372	5,3	15,4	−1		
`Churn`	4372	0,333	0,471	0	1	Déséquilibre 67/33

Valeurs sentinelles critiques détectées :

- `SatisfactionScore` $\in \{-1, 0, 99\}$: l'échelle logique est 1–5.
- `SupportTicketsCount` $\in \{-1, 999\}$: une valeur de 999 tickets support est physiquement impossible

4.5 Analyse des valeurs manquantes

L'analyse des valeurs manquantes révèle une problématique importante dans le dataset. Certaines variables présentent un taux de valeurs manquantes significatif, notamment une variable liée à l'âge des clients avec environ 30% de données absentes.

D'autres variables présentent un faible taux de valeurs manquantes, souvent lié à des cas particuliers comme les clients n'ayant effectué qu'un seul achat, rendant certains indicateurs temporels non calculables.vgDaysBetweenPurchases : 1,8%

4.6 Analyse de la variable cible — Churn

L'analyse de la variable cible est une étape fondamentale : la distribution du Churn détermine si les classes sont équilibrées et, dans le cas contraire, quelle technique de rééquilibrage appliquer.

Résultat de l'analyse de la distribution du churn obtenu est illustrée dans la figure

Déséquilibre de classes détecté : le ratio est de 67 % de clients fidèles contre 33 % de clients churnés, soit environ **2 clients fidèles pour 1 churné**. Ce déséquilibre est modéré mais suffisant pour biaiser un modèle de classification.

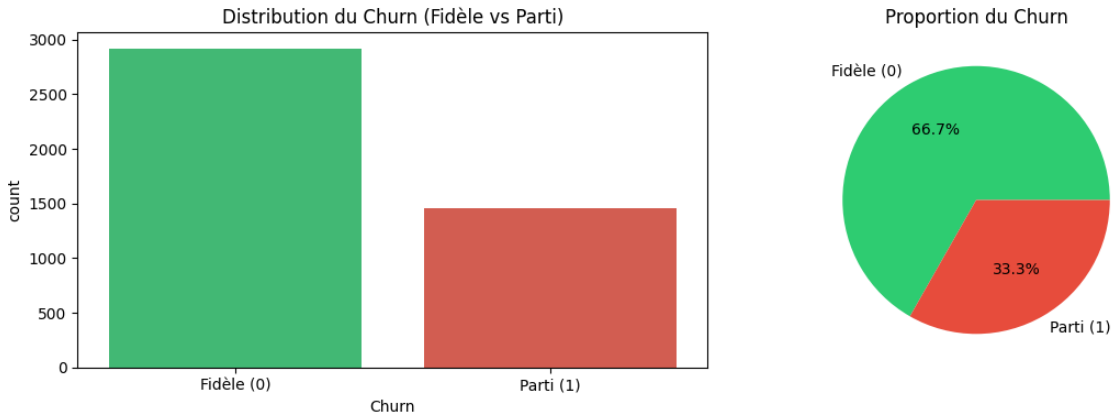

FIGURE 1 – **Distribution de la variable cible Churn** — déséquilibre modéré entre classes

4.7 Identification des features à traitement spécifique

Outre les anomalies numériques, l'exploration des variables catégorielles et textuelles révèle des besoins de transformation critiques :

— **NewsletterSubscribed :** Cette variable présente une variance nulle avec 100 % de valeurs constantes (`"Yes"`).

— **Registration Date :** Les formats sont inconsistants, mélangeant les standards UK (`DD/MM/YY`) et ISO (`YYYY-MM-DD`).

— **Last LoginIP :**L'adresse IP ne pouvant être utilisée telle quelle par les algorithmes.

— **Country :** Avec plus de 37 pays différents, cette variable à haute cardinalité nécessite un encodage spécifique .

4.8 Analyse de corrélation et Multicolinéarité

L'analyse des relations entre les 52 features est cruciale pour la stabilité des modèles linéaires. Une forte corrélation (proche de 1 ou -1) entre deux variables indique une redondance de l'information[cite : 114].

4.9 Analyse de Corrélation et Multicolinéarité

L'analyse des relations linéaires entre les 52 variables est indispensable pour garantir la stabilité des modèles. Deux outils complémentaires sont utilisés :

— **Matrice de corrélation :** Elle mesure la relation linéaire entre deux variables, avec un coefficient compris entre -1 et 1. Une attention particulière est portée aux couples dont la corrélation est supérieure à 0.8, seuil à partir duquel la suppression d'une variable redondante est préconisée.

— **VIF (Variance Inflation Factor) :** Alors que la corrélation traite les paires, le VIF détecte si une feature peut être prédite par une combinaison linéaire des autres. On considère qu'il y a une **multicolinéarité sévère** lorsque :

$$VIF_i = \frac{1}{1 - R_i^2} > 10$$

où R_i^2 est le coefficient de détermination de la régression de la feature i sur toutes les autres.

Constats sur le dataset : Des risques de multicolinéarité élevée ont été identifiés entre `MonetaryTotal` et `MonetaryAvg`, ainsi qu'entre `TotalQuantity` et `AvgQtyPerTrans`.

5 Préparation des Données (Data Preprocessing)

Introduction et vue d'ensemble du pipeline

Le prétraitement des données est l'étape qui transforme le dataset brut — imparfait, hétérogène, contenant des erreurs — en un jeu de données propre, structuré et exploitable par les algorithmes de machine learning.

Règle fondamentale — No Data Leakage : toutes les transformations statistiques (médiane, seuils, paramètres du scaler, axes ACP) sont calculées *uniquement* sur le train set, puis appliquées telles quelles sur le test set. La variable cible `Churn` n'est jamais normalisée ni transformée.

5.1 Suppression des features inutiles

La première étape du prétraitement consiste à supprimer les variables non pertinentes afin d'éviter les biais et améliorer la généralisation du modèle.

Les colonnes suivantes ont été retirées :

- `ChurnRiskCategory` et `AccountStatus` : sources de **data leakage**, car elles contiennent des informations directement liées ou postérieures à la cible `Churn`.
- `CustomerID` : identifiant unique sans pouvoir prédictif.
- `NewsletterSubscribed` : variable à variance nulle, donc non informative.

La suppression de `ChurnRiskCategory` est particulièrement importante, car elle est dérivée directement de la variable cible. La conserver conduirait à un apprentissage biaisé et à des performances artificiellement élevées lors de l'entraînement, mais non reproductibles en conditions réelles.

5.2 Imputation des valeurs manquantes

L'imputation consiste à remplacer les valeurs manquantes (`NaN`) par des valeurs substitutives calculées à partir des données disponibles. Deux variables présentent des valeurs manquantes, chacune avec une cause distincte et donc une stratégie d'imputation différente.

- **Imputation de `Age` — 30 % de valeurs manquantes**
  - Méthode : imputation par la **médiane**
  - Justification : la médiane est robuste aux valeurs extrêmes, contrairement à la moyenne
  - Avantage : elle représente mieux le client typique (49 ans) sans être influencée par les outliers
- **Imputation de `AvgDaysBetweenPurchases` — 1,8 % de valeurs manquantes**
  - Nature du problème : variable indéfinie pour les clients n'ayant effectué qu'un seul achat
  - Méthode : remplacement par **0**
  - Justification : un client "one-shot" n'a aucun intervalle entre achats, donc 0 est une valeur sémantiquement correcte

5.3 Traitement des valeurs aberrantes (outliers)

- **Correction de `SatisfactionScore`**
  - Problème : valeurs hors intervalle $\{-1, 0, 99\}$ ne respectant pas l'échelle 1–5
  - Interprétation : erreurs de saisie, clients non interrogés ou valeurs par défaut système
  - Stratégie : remplacement par la **médiane des valeurs valides (1–5)**
  - Avantage : évite la contamination du calcul par des outliers
- **Correction de `SupportTicketsCount`**
  - Problème : présence de valeurs négatives (-1) et extrêmes (999)

- Interprétation : codes système ou valeurs sentinelles
- Stratégie : **écrêtage au 99e percentile**
- Avantage : limite l'impact des valeurs aberrantes sans suppression brutale

5.4 Feature Engineering

Le *feature engineering* consiste à créer de nouvelles variables à partir des variables existantes afin de capturer des informations comportementales non visibles dans les données brutes.

- **Parsing de `RegistrationDate` → extraction de features temporelles**

  La variable `RegistrationDate` est convertie en `datetime` puis utilisée pour extraire des features temporelles.

Nouvelle feature	Signification	Information capturée
`RegYear`	Année d'inscription	Ancienneté du client sur la plateforme
`RegMonth`	Mois d'inscription	Saisonnalité des acquisitions clients
`RegDay`	Jour du mois	Cycles de comportement d'achat
`RegWeekday`	Jour de la semaine	Profil semaine vs weekend

- **Extraction depuis `LastLoginIP` → `Is_Private_IP`**
  - Création d'une variable binaire indiquant si l'adresse IP est privée ou publique
  - Permet de détecter des comportements de connexion différents (VPN, entreprise, domicile)
- **Création de features comportementales (RFM)**

  Trois variables sont construites selon le modèle **RFM**.

Feature créée	Formule	Interprétation métier
`MonetaryPerDay`	$\frac{MonetaryTotal}{Recency+1}$	Intensité de dépense récente ; indicateur de fidélité active
`AvgBasketValue`	$\frac{MonetaryTotal}{Frequency}$	Valeur moyenne par achat ; distingue petits et gros acheteurs
`TenureRatio`	$\frac{Recency}{CustomerTenureDays}$	Indicateur de désengagement client (risque de churn)

5.5 Encodage des variables catégorielles

Les algorithmes de machine learning opèrent exclusivement sur des données numériques. Les variables catégorielles doivent donc être transformées en représentations numériques avant d'être utilisées par un modèle. Le choix de la méthode d'encodage dépend de la nature ordinale ou nominale des variables.

Le **One-Hot Encoding (OHE)** transforme chaque modalité en une colonne binaire (0 ou 1). Il est adapté aux variables **nominales** (sans ordre intrinsèque) telles que `Gender`, `Region` ou `CustomerType`.

Le paramètre `drop_first=True` permet d'éviter le *dummy variable trap*, c'est-à-dire la multicolinéarité parfaite entre les colonnes binaires.

Impact dimensionnel : L'encodage OHE a augmenté le nombre de variables de 51 à **112 colonnes** (+61 colonnes binaires). Cette explosion dimensionnelle est attendue et sera traitée par une **ACP**

(Analyse en Composantes Principales) afin de conserver 95% de la variance et réduire la redondance entre variables.

Comparaison des techniques d'encodage

Technique	Type de variable	Avantage	Utilisé ici
One-Hot Encoding	Nominale (sans ordre)	Pas d'ordre artificiel entre modalités	Oui (toutes)
Ordinal Encoding	Ordinale (avec ordre)	Préserve la hiérarchie (Low < Medium < High)	Possible (`SpendingCat`)
Target Encoding	Haute cardinalité	Réduit l'explosion dimensionnelle	Alternative (`Country`)
Label Encoding	Binaire (2 modalités)	Simple (une seule colonne)	—

5.6 Normalisation — StandardScaler

La normalisation ramène toutes les features numériques à une même échelle de variation. Sans elle, `MonetaryTotal` (centaines de livres sterling) écraserait numériquement `ReturnRatio` (0–1), biaisant les distances et les poids appris.

Principe mathématique

$$z_i = \frac{x_i - \mu}{\sigma} \qquad \Rightarrow \qquad \mathbb{E}[z] = 0, \quad Var(z) = 1 \tag{1}$$

5.7 Analyse de corrélation et multicolinéarité

La multicolinéarité — corrélation forte entre plusieurs features — est problématique pour les modèles linéaires : elle rend les coefficients instables et l'interprétation difficile. Deux outils complémentaires sont utilisés.

Matrice de corrélation de Pearson

$$r_{XY} = \frac{Cov(X,\, Y)}{\sigma_X \cdot \sigma_Y} \;\in\; [-1,\; 1] \tag{2}$$

Le seuil de $|r| > 0{,}8$ signale une corrélation forte. L'observation des données brutes révèle de fortes corrélations attendues entre : `MonetaryTotal` et `MonetaryAvg` (mêmes transactions financières), `Frequency` et `UniqueInvoices` (redondants par construction), `UniqueProducts` et `UniqueDescriptions` (quasi-identiques avec $r \approx 0{,}99$).

VIF — Variance Inflation Factor

Le VIF détecte la multicolinéarité *multiple* : si une feature peut être prédite par une combinaison linéaire de plusieurs autres.

$$VIF_i = \frac{1}{1 - R_i^2} \qquad VIF > 10 \;\Rightarrow\; multicolinéaritésévère \tag{3}$$

où R_i^2 est le coefficient de détermination de la régression de la feature i sur toutes les autres. Dans ce dataset, les groupes {`Monetary*`}, {`Frequency`, `UniqueInvoices`, `TotalTransactions`} et {`UniqueProducts`, `UniqueDescriptions`} présentent des VIF très élevés (souvent > 100).

Décision de conception : les features à forte multicolinéarité ne sont pas supprimées manuellement. L'ACP (section suivante) résout ce problème de façon systématique : les composantes principales sont orthogonales par construction, éliminant toute multicolinéarité résiduelle.

5.8 Réduction dimensionnelle — ACP

L'ACP résout simultanément trois problèmes identifiés : la multicolinéarité (VIF > 10 pour de nombreuses features), la forte dimensionnalité (112 colonnes après OHE) et le bruit des features peu informatives.

Principe mathématique

L'ACP cherche une projection linéaire $\mathbf{Z} = \mathbf{X}\mathbf{W}$ où les axes de projection (vecteurs propres) sont ordonnés par variance décroissante :

$$\Sigma \mathbf{w}_k = \lambda_k \mathbf{w}_k \quad (\lambda_1 \geq \lambda_2 \geq \ldots) \qquad \Sigma = \frac{\mathbf{X}^\top \mathbf{X}}{n-1} \tag{4}$$

Les composantes sont **orthogonales par construction** — la multicolinéarité est structurellement éliminée.

5.9 Séparation Train / Test stratifiée (80/20)

```python
from sklearn.model_selection import train_test_split

# Separation features / cible
X = df_encoded.select_dtypes(include=[np.number]).drop(
        columns=['Churn'], errors='ignore')
y = df_encoded['Churn']

print(f"Dimensions de X : {X.shape}")
print(f"Distribution de y :\n{y.value_counts()}")

# Split stratifie : la proportion Churn/Actif est preservee
# dans le train set ET dans le test set
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size   = 0.20,    # 20 % pour le test
    random_state= 42,      # Reproductibilite
    stratify    = y        # Conserver la proportion du Churn
)

print(f"\nX_train : {X_train.shape}  |  X_test : {X_test.shape}")
print(f"Proportion Churn y_train : {y_train.mean():.3f}")
print(f"Proportion Churn y_test  : {y_test.mean():.3f}")
```

Listing 2 – Split stratifié 80/20 avec `train_test_split`

```
Dimensions de X : (4372, 111)
Distribution de y :
0    2918
1    1454

X_train : (3497, 111)  |  X_test : (875, 111)
Proportion Churn y_train : 0.333
Proportion Churn y_test  : 0.332
```

Importance de `stratify=y` : sans stratification, un split aléatoire pourrait par malchance regrouper davantage de clients churnés dans le train ou dans le test. La stratification garantit que le ratio 67 %/33 % est identique dans les deux ensembles — condition nécessaire pour que les métriques d'évaluation sur le test soient représentatives de la vraie distribution des données.

5.10 Rééquilibrage par SMOTE

L'EDA a révélé un déséquilibre modéré (67 % fidèles / 33 % churnés). La technique **SMOTE** (*Synthetic Minority Over-sampling Technique*) corrige ce déséquilibre en générant de nouveaux exemples synthétiques de la classe minoritaire (Churn = 1), par interpolation entre des exemples minoritaires existants et leurs k plus proches voisins dans l'espace des features.

```python
from imblearn.over_sampling import SMOTE

# IMPORTANT : SMOTE s'applique UNIQUEMENT sur X_train / y_train
# Le test set doit rester intact (vraie distribution de production)
smote = SMOTE(random_state=42)
X_train_res, y_train_res = smote.fit_resample(X_train, y_train)

print("--- Resultats du reequilibrage ---")
print("Repartition avant SMOTE :")
print(y_train.value_counts(normalize=True).round(3))
print("\nRepartition apres SMOTE :")
print(y_train_res.value_counts(normalize=True).round(3))
print(f"\nDimensions : X_train_res = {X_train_res.shape}")
```

Listing 3 – Application de SMOTE sur le train set uniquement

```
--- Resultats du reequilibrage ---
Repartition avant SMOTE :
0    0.667
1    0.333

Repartition apres SMOTE :
0    0.500
1    0.500

Dimensions : X_train_res = (4668, 111)
```

SMOTE sur le train set uniquement — règle absolue : appliquer SMOTE sur l'ensemble des données *avant* le split constituerait un data leakage : les exemples synthétiques générés à partir d'observations du test se retrouveraient dans le train, et le modèle évaluerait des données qu'il a « vues » lors de l'entraînement. Le test set doit impérativement représenter la distribution réelle de production : non rééquilibré, avec 33 % de churnés.

5.11 Récapitulatif du prétraitement

#	Action réalisée	Variables concernées	Impact mesurable
1	Suppression (leakage)	`ChurnRiskCategory`, `AccountStatus`	52 → 50 colonnes
2	Suppression (inutile)	`CustomerID`, `Newsletter...`	50 → 48 colonnes
3	Imputation médiane	`Age` (1 311 NaN)	0 NaN restants
4	Imputation constante	`AvgDaysBetween...` (79 NaN)	0 NaN restants
5	Correction outliers	`SatisfactionScore`, `SupportTicketsCount`	Plages valides : [1–5] et [0–8]
6	Parsing + Feature Eng.	`RegistrationDate` → 4 features	+4 colonnes temporelles
7	Feature Engineering	`LastLoginIP` → `Is_Private_IP`	Donnée brute rendue exploitable
8	Feature Engineering	`MonetaryPerDay`, `AvgBasketValue`, `TenureRatio`	+3 indicateurs comportementaux
9	One-Hot Encoding	Variables catégorielles (27 colonnes)	51 → 112 colonnes
10	Split Train/Test stratifié	80 % train / 20 % test	3 497 / 875 observations
11	SMOTE (train set seul)	`Churn` (67 % / 33 %)	3 497 → 4 668 observations (train)
Dataset original		**4 372 lignes × 52 colonnes**	
Train set final (SMOTE)		**4 668 lignes × 111 colonnes — classes équilibrées**	
Test set final		**875 lignes × 111 colonnes — distribution réelle**	

Bilan du prétraitement : en 11 étapes séquentielles, le dataset brut de 4 372 observations et 52 colonnes hétérogènes a été transformé en un jeu de données propre, encodé et rééquilibré prêt pour la modélisation. Le train set final contient 4 668 observations (grâce à SMOTE) avec des classes parfaitement équilibrées (50/50), tandis que le test set conserve la distribution réelle de production (67 % / 33 %) pour une évaluation honnête des performances.

6 Modélisation et Entraînement

L'entraînement des modèles est réalisé par le module `src/train_model.py`. Deux algorithmes de classification supervisée sont comparés : la **Régression Logistique** et le **Random Forest Classifier**.

6.1 Régression Logistique

Principe mathématique

$$P(y = 1|x) = \sigma(w^T x + b) = \frac{1}{1 + e^{-(w^T x + b)}}$$

La classe est prédite selon un seuil de 0.5 :

$$P(y = 1|x) > 0.5 \Rightarrow Churn = 1$$

Hyperparamètres testés

#	Hyperparamètre	Valeurs testées	Rôle
1	C (régularisation)	0.01, 0.1, 1, 10, 100	Contrôle la pénalisation (évite le sur-apprentissage)
2	solver	lbfgs, liblinear	Algorithme d'optimisation
3	max_iter	100, 500, 1000	Nombre maximal d'itérations
4	penalty	l1, l2	Type de régularisation

Avantages et limites

- **Avantages :**
  - Modèle interprétable et rapide
  - Probabilités bien calibrées
  - Bonne généralisation avec régularisation
- **Limites :**
  - Suppose une relation linéaire
  - Performances limitées sur données complexes

6.2 Random Forest Classifier

Principe — Ensemble Learning

$$Prédiction = Mode(Arbre_1(x), \ldots, Arbre_n(x))$$

Hyperparamètres testés

#	Hyperparamètre	Valeurs testées	Rôle
1	n_estimators	100, 200, 500	Nombre d'arbres
2	max_depth	None, 5, 10, 20	Profondeur maximale
3	min_samples_split	2, 5, 10	Seuil de division
4	min_samples_leaf	1, 2, 4	Taille minimale d'une feuille
5	max_features	sqrt, log2	Nombre de variables par split

Avantages et limites

- **Avantages :**
  - Très performant sur données non linéaires
  - Robuste aux outliers
  - Permet l'analyse d'importance des variables
- **Limites :**
  - Moins interprétable
  - Coût de calcul plus élevé

6.3 Optimisation — GridSearchCV

Principe

GridSearchCV explore toutes les combinaisons d'hyperparamètres avec validation croisée.

Configuration utilisée

#	Paramètre	Valeur
1	Validation croisée	5-fold
2	Métrique	F1-score
3	Parallélisation	Tous les cœurs (n_jobs = -1)

Validation croisée

Les données sont divisées en 5 plis : 4 pour l'entraînement et 1 pour la validation, répété 5 fois. Le score final est la moyenne des performances.

7 Évaluation des Modèles

Dans cette section, les performances des modèles de classification sont évaluées à l'aide de métriques adaptées au problème de prédiction du *churn*, caractérisé par un déséquilibre des classes.

7.1 Métriques d'évaluation

Métrique	Définition	Interprétation
Accuracy	$\frac{TP+TN}{TP+TN+FP+FN}$	Mesure globale des prédictions correctes (peut être biaisée si déséquilibre)
Precision	$\frac{TP}{TP+FP}$	Fiabilité des clients prédits churnés
Recall	$\frac{TP}{TP+FN}$	Capacité à détecter les vrais churners (critique métier)
F1-Score	$2 \times \frac{P \times R}{P+R}$	Compromis entre Precision et Recall (métrique principale)
AUC-ROC	Aire sous la courbe ROC	Capacité du modèle à distinguer les deux classes

Choix métrique : Le **F1-Score** est privilégié, car il équilibre précision et rappel. Dans le contexte du churn, les **faux négatifs** sont critiques, d'où l'importance du Recall.

7.2 Comparaison des modèles

Modèle	Accuracy	Precision	Recall	F1	AUC
Régression Logistique	0.8869	0.7981	**0.8832**	**0.8385**	0.9504
Random Forest	**0.8926**	**0.8774**	0.7869	0.8297	**0.9524**
Meilleur modèle retenu	**Régression Logistique (meilleur F1-Score)**				

7.3 Analyse des performances

L'analyse comparative met en évidence les points suivants :

- Le **Random Forest** offre les meilleures performances globales en termes d'Accuracy, Precision et AUC-ROC.
- La **Régression Logistique** présente un **Recall plus élevé**, ce qui permet de mieux détecter les clients à risque de churn.
- Le **F1-Score**, métrique principale, est supérieur pour la Régression Logistique, indiquant un meilleur équilibre global.

Ainsi, le modèle de **Régression Logistique** est retenu, car il minimise le risque métier lié aux faux négatifs tout en maintenant de bonnes performances globales.

7.4 Matrice de confusion

	Prédit Actif (0)	Prédit Churn (1)
Réel Actif (0)	TN	FP
Réel Churn (1)	FN	TP

Point clé : Les faux négatifs (FN) sont les erreurs les plus critiques car ils correspondent aux clients churnés non détectés.

7.5 Importance des variables

Le modèle **Random Forest** permet d'estimer l'importance des variables via la réduction de l'impureté de Gini. Cette analyse identifie les variables les plus influentes dans la prédiction du churn.

```
importances = best_rf_model.feature_importances_
```

Ces résultats peuvent être exploités pour orienter les stratégies de rétention client et améliorer la prise de décision.

8 Déploiement de l'Application Flask

La dernière phase du projet consiste à rendre le modèle entraîné accessible via une interface web interactive, développée avec le framework **Flask** (Python). L'application suit le patron de conception **MVC (Modèle-Vue-Contrôleur)**, garantissant une séparation claire des responsabilités et une architecture maintenable.

8.1 Architecture de l'Application

Composant	Fichier	Rôle
Modèle (Model)	`src/predict.py`	Classe `ChurnPredictor` — chargement des artefacts `.pkl` et pipeline d'inférence
Vue (View)	`app/templates/index.html`	Interface utilisateur HTML/CSS responsive
Contrôleur	`app/routes.py`	Blueprint Flask — définition des endpoints HTTP
Factory	`app/__init__.py`	Initialisation de l'application Flask via le patron Application Factory
Point d'entrée	`run.py`	Lancement du serveur de développement

Le diagramme de flux applicatif structure les interactions entre l'utilisateur et les différents modules du système :

```
Utilisateur (navigateur)
        |
        V
    run.py  ------------------------ port 5000
        |          create_app()
        V
  app/__init__.py  ---- enregistre ---- Blueprint (routes.py)
                                                |
                  +---------------------------+
                  |    /           /predict    |
                  V                V
          index.html          ChurnPredictor
                              (predict.py)
                                   |
                        preprocessor_pipeline.pkl
                        best_model.pkl
                        features_meta.pkl
```

8.2 Point d'Entrée et Configuration (run.py)

Le point d'entrée de l'application configure le `sys.path` pour rendre accessibles les modules `src/` depuis n'importe quel niveau, puis instancie l'application :

```python
import os, sys

# Ajout de la racine du projet et du dossier src dans le path Python
sys.path.append(os.path.abspath(os.path.dirname(__file__)))
sys.path.append(os.path.join(os.path.abspath(os.path.dirname(__file__)), 'src'))

from app import create_app

app = create_app()

```

```
if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0', port=5000)
```

8.3 Patron Application Factory (app/___init___.py)

L'application est créée via la fonction suivante , un patron recommandé par Flask pour les projets modulaires permettant d'isoler la logique de routage.

```
from flask import Flask

def create_app():
    app = Flask(__name__)

    from app.routes import main_bp
    app.register_blueprint(main_bp)

    return app
```

8.4 Module de Prédiction — ChurnPredictor (src/predict.py)

La classe `ChurnPredictor` encapsule l'intégralité de la logique d'inférence en chargeant les trois artefacts sérialisés :

Artefact	Fichier	Contenu
Pipeline	`models/preprocessor_pipeli`	Imputation + StandardScaler + OneHotEncoder + PCA
Métadonnées	`models/features_meta.pkl`	Colonnes numériques/catégoriques, colonnes exclues
Meilleur modèle	`models/best_model.pkl`	Modèle sélectionné (LR ou RF)

8.4.1 Méthode prepare_input()

```
def prepare_input(self, raw_df):
    df_clean = handle_outliers(raw_df)      # Correction des outliers
    df_clean = feature_engineering(df_clean) # Feature engineering

    # Colonnes manquantes -> valeurs par defaut coherentes
    for col in self.meta['num_cols']:
        if col not in df_clean.columns:
            df_clean[col] = 0
    for col in self.meta['cat_cols']:
        if col not in df_clean.columns:
            df_clean[col] = 'Unknown'

    # Suppression des colonnes exclues (correlation > 0.85)
    dropped = self.meta['dropped_cols']
    df_clean = df_clean.drop(columns=[c for c in dropped if c in df_clean.columns
    ])

    return df_clean
```

8.5 API REST — Endpoints (app/routes.py)

```
@main_bp.route('/predict', methods=['POST'])
def predict():
    data = request.form.to_dict()
```

```python

    # Conversion des champs numeriques
    for key in NUMERIC_FIELDS:
        raw = data.get(key, '').strip()
        data[key] = float(raw) if raw else 0.0

    df = pd.DataFrame([data])
    preds, probs = predictor.predict(df)

    churn_class = int(preds[0])
    churn_prob  = float(probs[0])

    status = "Risque de Churn" if churn_class == 1 else "Client Fidele"

    return render_template('index.html',
        prediction=status,
        probability=round(churn_prob * 100, 1),
        form_data=data
    )
```

8.6 Sérialisation des Artefacts avec joblib

Le module `src/utils.py` centralise la logique de persistance. `joblib` est privilégié par rapport à `pickle` pour son efficacité avec les objets Scikit-Learn contenant des tableaux NumPy volumineux.

```python
import joblib

# Sauvegarde (post-entrainement)
joblib.dump(pipeline,   'models/preprocessor_pipeline.pkl')
joblib.dump(best_model, 'models/best_model.pkl')
joblib.dump(meta,       'models/features_meta.pkl')

# Chargement (demarrage Flask)
pipeline   = joblib.load('models/preprocessor_pipeline.pkl')
best_model = joblib.load('models/best_model.pkl')
meta       = joblib.load('models/features_meta.pkl')
```

8.7 Lancement de l'Application

Pour déployer l'application localement, exécutez les commandes suivantes dans l'ordre :

```bash
# Etape 1 : Pretraitement des donnees
python src/preprocessing.py

# Etape 2 : Entrainement et selection du modele
python src/train_model.py

# Etape 3 : Lancement du serveur Flask
python run.py
```

```
* Serving Flask app 'app'
* Debug mode: on
* Running on http://0.0.0.0:5000
```

Conclusion Générale

Ce projet a permis de concevoir, développer et déployer une solution complète de prédiction du churn client basée sur des techniques de Machine Learning. L'objectif principal était d'anticiper le départ des clients afin d'aider les entreprises à mettre en place des stratégies de rétention efficaces et orientées données.

Dans un premier temps, un travail rigoureux de préparation des données a été effectué, incluant le nettoyage, le traitement des valeurs aberrantes, l'encodage des variables catégorielles ainsi que la normalisation des variables numériques. Cette étape a été déterminante pour garantir la qualité des données utilisées par les modèles.

Ensuite, plusieurs algorithmes de classification ont été étudiés et comparés, notamment la Régression Logistique et la Forêt Aléatoire. L'optimisation des hyperparamètres à l'aide de la validation croisée (GridSearchCV) a permis d'améliorer significativement les performances des modèles. Le choix final s'est basé sur des métriques pertinentes telles que le F1-score, l'accuracy et le ROC-AUC, afin de tenir compte du déséquilibre des classes.

Par la suite, une phase d'évaluation approfondie a été réalisée, incluant l'analyse des matrices de confusion et l'étude de l'importance des variables, permettant une meilleure interprétation des résultats et une compréhension des facteurs influençant le churn client.

Enfin, le modèle retenu a été déployé sous forme d'une application web interactive développée avec le framework Flask. Cette application suit une architecture MVC claire et modulaire, facilitant la maintenance et l'évolution du système. L'intégration du modèle dans une API REST permet une utilisation en temps réel, rendant la solution directement exploitable par les utilisateurs finaux.

Ce projet met en évidence l'importance de l'intégration complète d'un pipeline de Machine Learning, allant de la collecte et préparation des données jusqu'au déploiement en production. Il démontre également la valeur ajoutée de l'intelligence artificielle dans la prise de décision stratégique des entreprises.

Perspectives d'amélioration : Plusieurs améliorations peuvent être envisagées pour enrichir ce travail, notamment l'intégration de modèles plus avancés tels que XGBoost ou les réseaux de neurones, l'utilisation de techniques d'explicabilité comme SHAP pour mieux interpréter les prédictions, ainsi que le déploiement sur une infrastructure cloud afin de rendre l'application accessible à grande échelle.